

BÁO CÁO THỰC HÀNH

Môn học: Bảo mật Web và Ứng dụng

Lab 5: Pentesting Android Applications

GVHD: Ngô Khánh Khoa

Nhóm: 11

1. THÔNG TIN CHUNG:

Lớp: NT213.P11.ANTT.2

STT	Họ và tên	MSSV	Email
1	Nguyễn Đặng Quỳnh Như	22521050	22521050@gm.uit.edu.vn
2	Phạm Minh Tân	22521310	22521310@gm.uit.edu.vn
3	Lê Minh Quân	22521181	22521181@gm.uit.edu.vn
4	Đặng Đức Tài	22521270	22521270@gm.uit.edu.vn

2. NỘI DUNG THỰC HIỆN:

STT	Nội dung	Tình trạng	Trang
1	Yêu cầu 1	100%	2
2	Yêu cầu 2	100%	5
3	Yêu cầu 3	100%	10
4	Yêu cầu 4	100%	15
5	Yêu cầu 5	100%	17
6	Yêu cầu 6	100%	19
Điểm tự đánh giá			10/10

Link video youtube: [Click here](#)

Phần bên dưới của báo cáo này là tài liệu báo cáo chi tiết của nhóm thực hiện

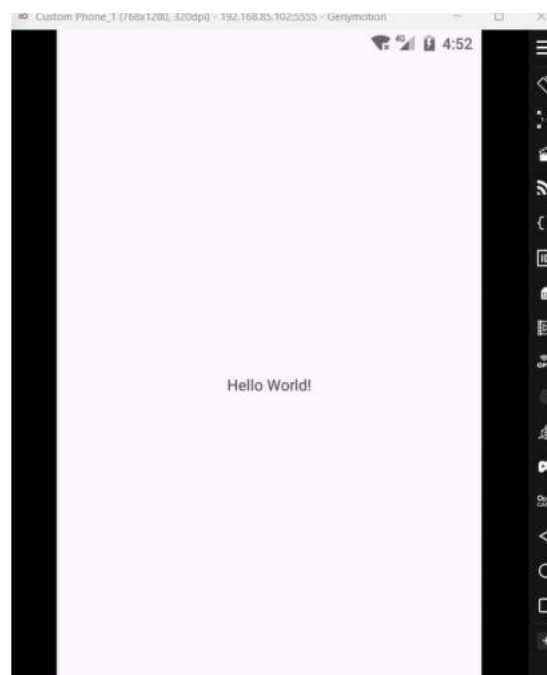
BÁO CÁO CHI TIẾT

Yêu cầu 1 Sinh viên tiếp tục sửa lỗi Broadcast Receivers.

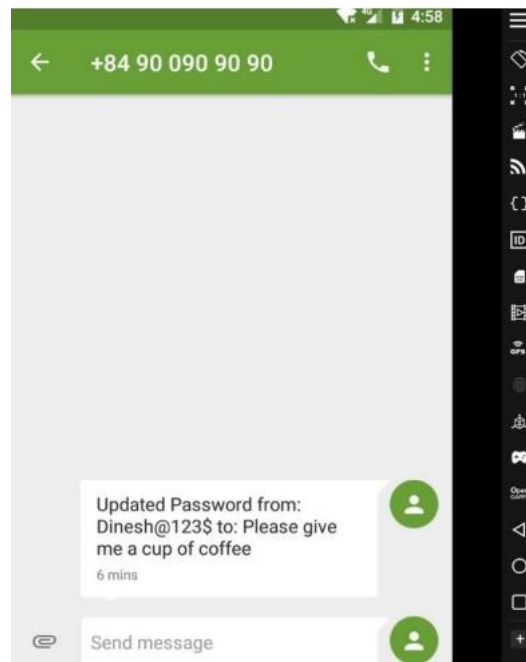
- Mã nguồn app tấn công

```
<? activity_main.xml MainActivity.java x
1 package com.example.cau7;
2
3 import android.os.Bundle;
4
5 import androidx.activity.EdgeToEdge;
6 import androidx.appcompat.app.AppCompatActivity;
7 import androidx.core.graphics.Insets;
8 import androidx.core.view.ViewCompat;
9 import androidx.core.view.WindowInsetsCompat;
10
11 import android.content.Intent;
12 public class MainActivity extends AppCompatActivity {
13
14     @Override
15     protected void onCreate(Bundle savedInstanceState) {
16         super.onCreate(savedInstanceState);
17         EdgeToEdge.enable(this);
18         setContentView(R.layout.activity_main);
19         Intent intent = new Intent(action: "theBroadcast");
20         intent.putExtra(name: "phonenumber", value: "+84900909090");
21         intent.putExtra(name: "newpass", value: "Please give me a cup of coffee");
22         sendBroadcast(intent);
23     }
24 }
```

- Build app thành công và tấn công BroastCast



- Không còn nhận được thông điệp từ attacker nữa



Yêu cầu 2 Sinh viên xây dựng ứng dụng Android gồm 3 giao diện chức năng chính:

- 1) Register - Đăng ký thông tin với ứng dụng (email, username, password).
- 2) Login - Đăng nhập vào ứng dụng (username, password).
- 3) Hiển thị thông tin người dùng (một lời chào có tên người dùng).

Hướng dẫn:

Các bước tạo một ứng dụng Android trong Android Studio.

Bước 1. Khởi động Android Studio.

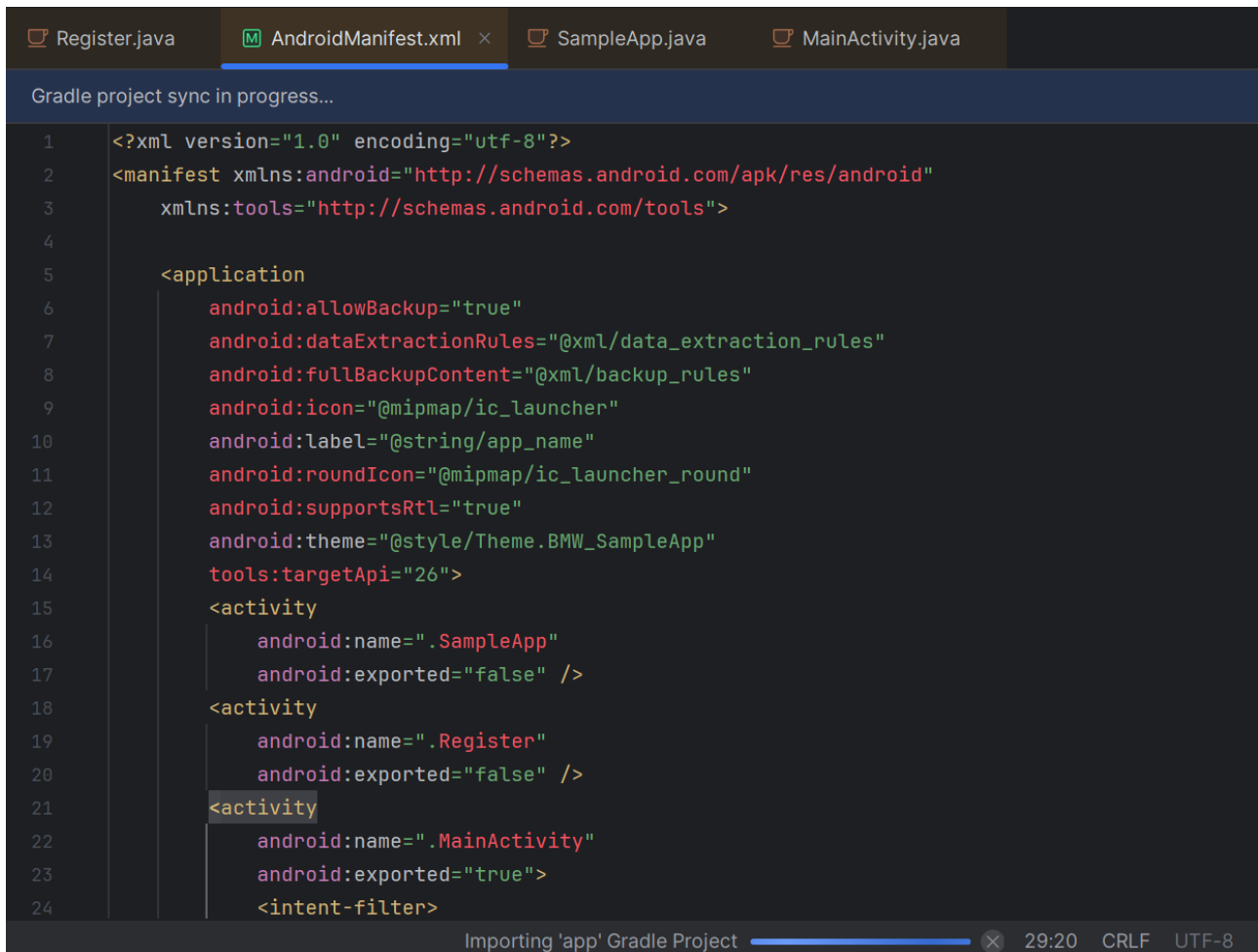
Bước 2. Chọn Start a new Android Studio project để tạo một Project mới.

Bước 3. Tạo một ứng dụng có chức năng đăng nhập bằng cách chọn Empty Activity dành cho Phone and Tablet như hình bên dưới. Nhấp Next để tiếp tục.

Bước 4. Điền các thông tin cần thiết cho ứng dụng Android như tên ứng dụng, tên package chứa nó, vị trí lưu mã nguồn, ngôn ngữ (thường là Java) và phiên bản API tối thiểu của thiết bị để chạy ứng dụng này. API tối thiểu càng thấp thì càng có nhiều thiết bị hỗ trợ được ứng dụng, như ở đây API 15 của Android 4.0.3 thì gần như 100% thiết bị sẽ chạy được. Chọn Finish để tạo project

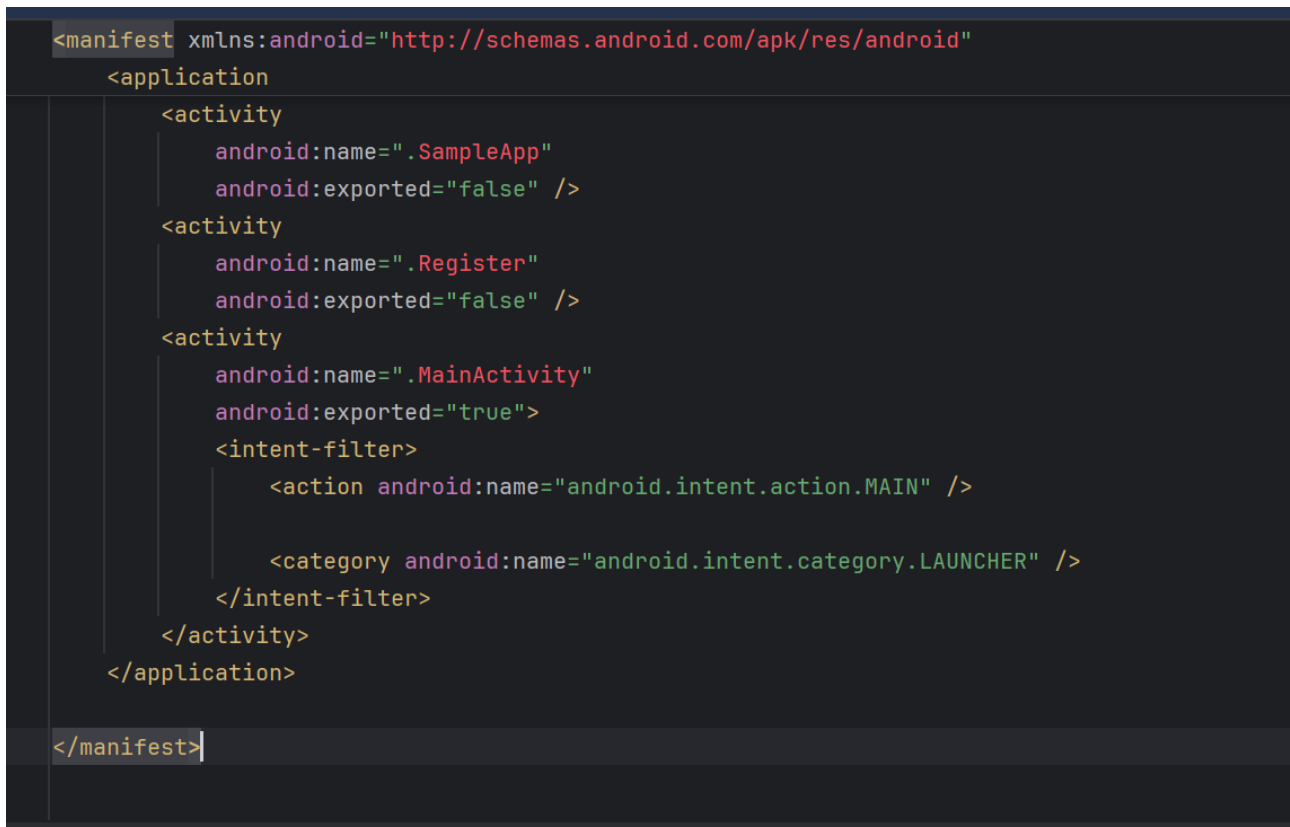
Các file quan trọng trong 1 project Android:

- manifests/AndroidManifest.xml: file cấu hình ứng dụng Android, khai báo cấu trúc thành phần ứng dụng, định nghĩa khai báo các quyền hạn của ứng dụng...



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:tools="http://schemas.android.com/tools">
4
5     <application
6         android:allowBackup="true"
7         android:dataExtractionRules="@xml/data_extraction_rules"
8         android:fullBackupContent="@xml/backup_rules"
9         android:icon="@mipmap/ic_launcher"
10        android:label="@string/app_name"
11        android:roundIcon="@mipmap/ic_launcher_round"
12        android:supportsRtl="true"
13        android:theme="@style/Theme.BMW_SampleApp"
14        tools:targetApi="26">
15        <activity
16            android:name=".SampleApp"
17            android:exported="false" />
18        <activity
19            android:name=".Register"
20            android:exported="false" />
21        <activity
22            android:name=".MainActivity"
23            android:exported="true">
24            <intent-filter>
```

Importing 'app' Gradle Project 29:20 CRLF UTF-8



```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <application
        <activity
            android:name=".SampleApp"
            android:exported="false" />
        <activity
            android:name=".Register"
            android:exported="false" />
        <activity
            android:name=".MainActivity"
            android:exported="true">
                <intent-filter>
                    <action android:name="android.intent.action.MAIN" />

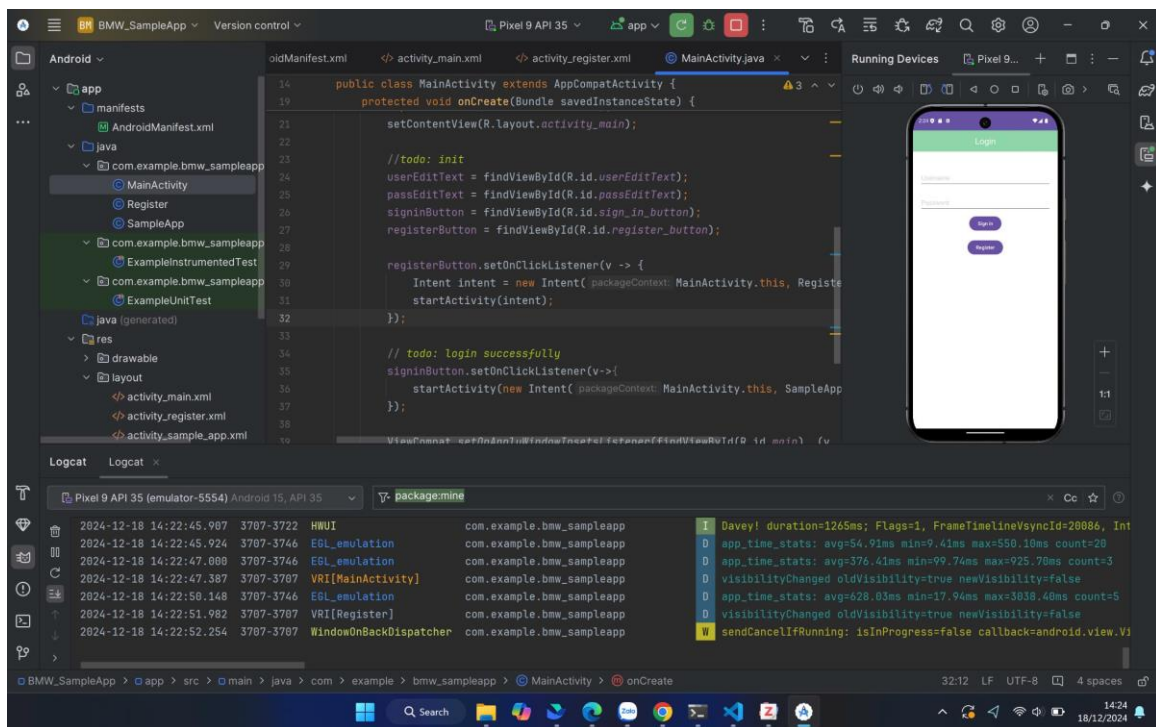
                    <category android:name="android.intent.category.LAUNCHER" />
                </intent-filter>
            </activity>
        </application>
    </manifest>
```

- Các file java của các activity: định nghĩa các function thực hiện chức năng của activity.

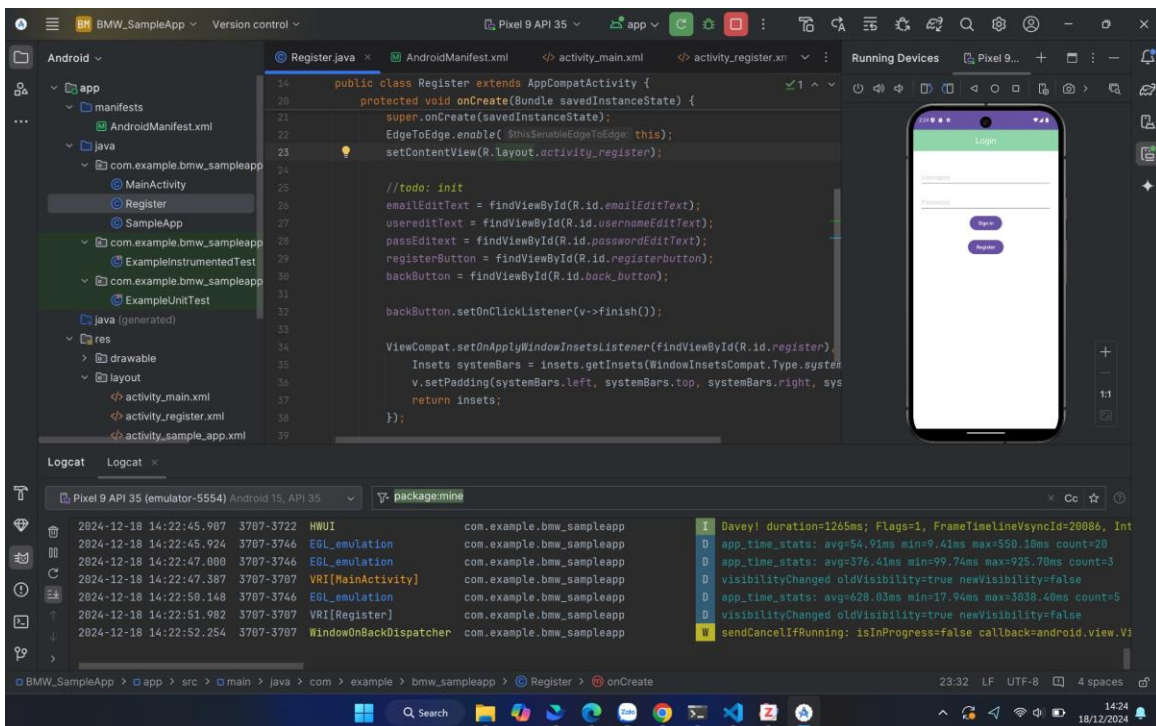
Lab 2: Tổng quan các lỗi hỏng web thường gặp (P2)



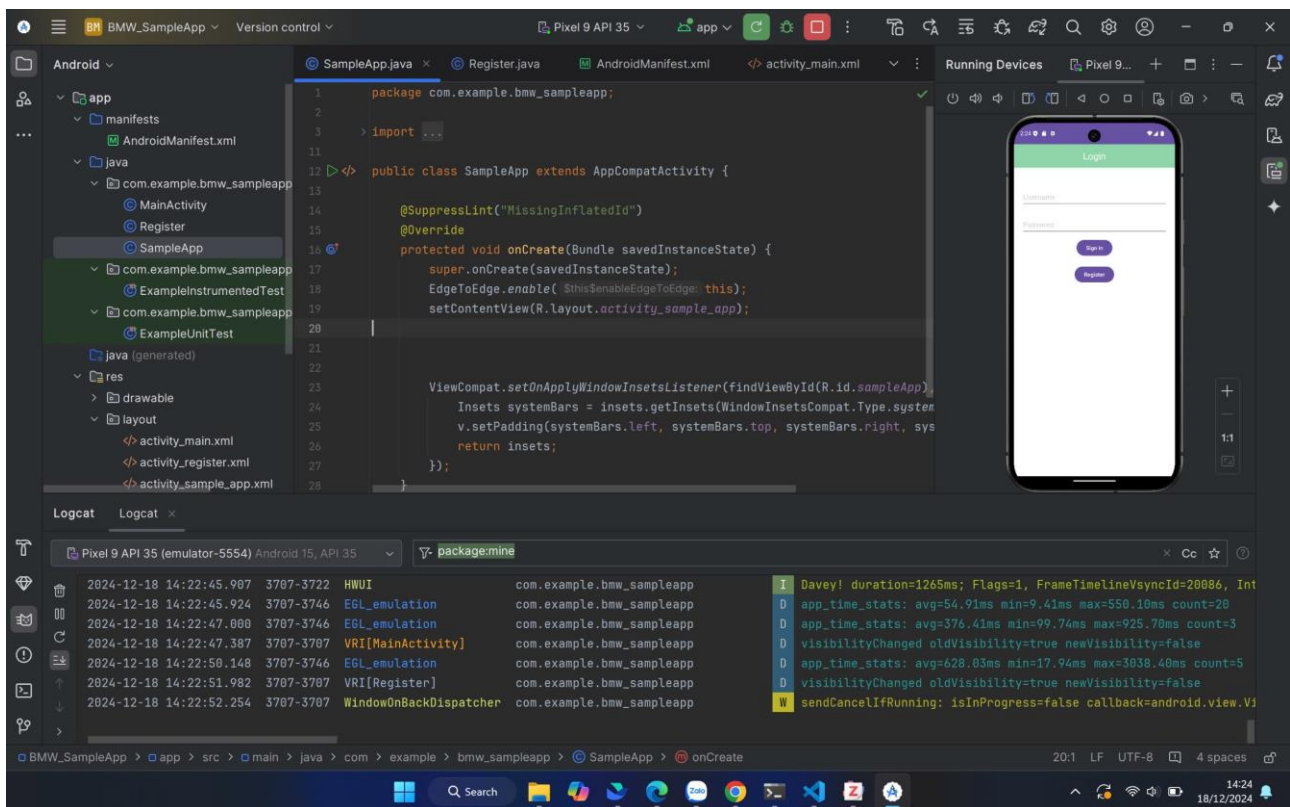
Main activity:



Register:



Sample app:

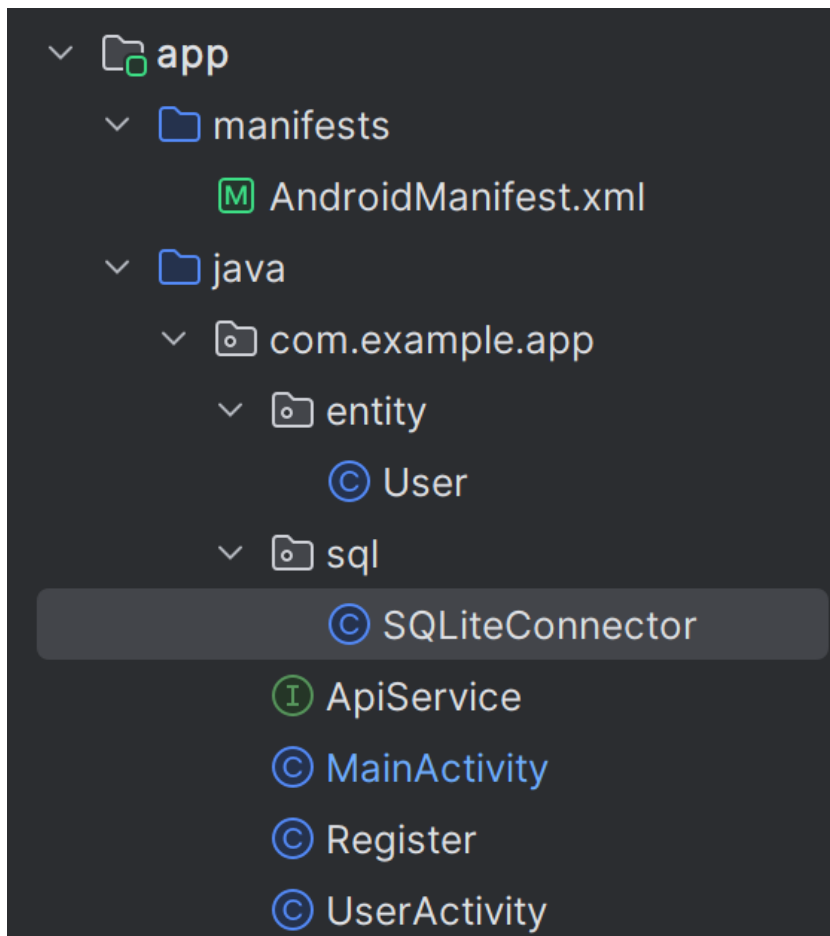


- Các file layout xml của các activity: định nghĩa giao diện của các activity. Trong Android Studio có tùy chọn Design cho phép kéo thả các thành phần hoặc Text để thêm các thành phần bằng mã nguồn XML.

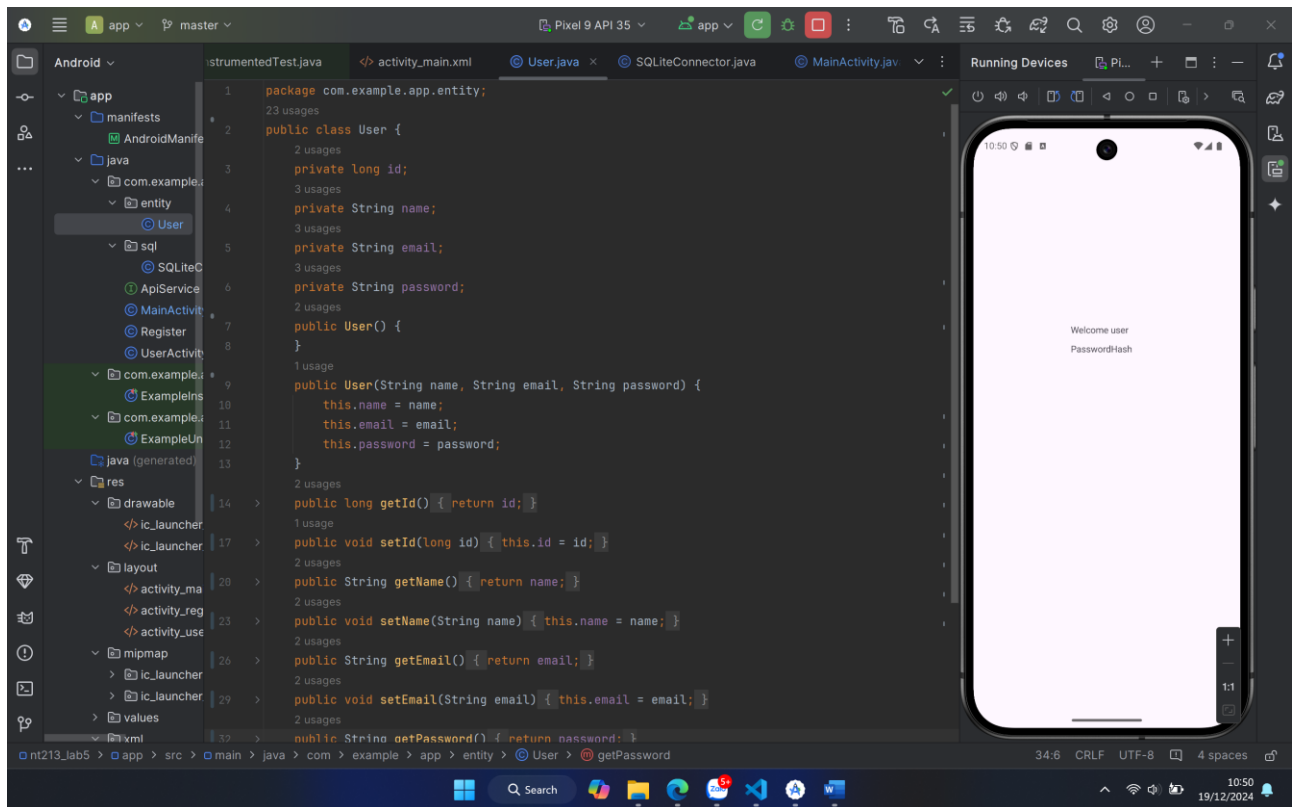
Yêu cầu 3 Sinh viên viết mã nguồn Java cho chức năng đăng nhập và đăng ký, sử dụng tập tin **SQLiteConnector** được giảng viên cung cấp để thực hiện kết nối đến cơ sở dữ liệu SQLite với các yêu cầu bên dưới.

Bài làm:

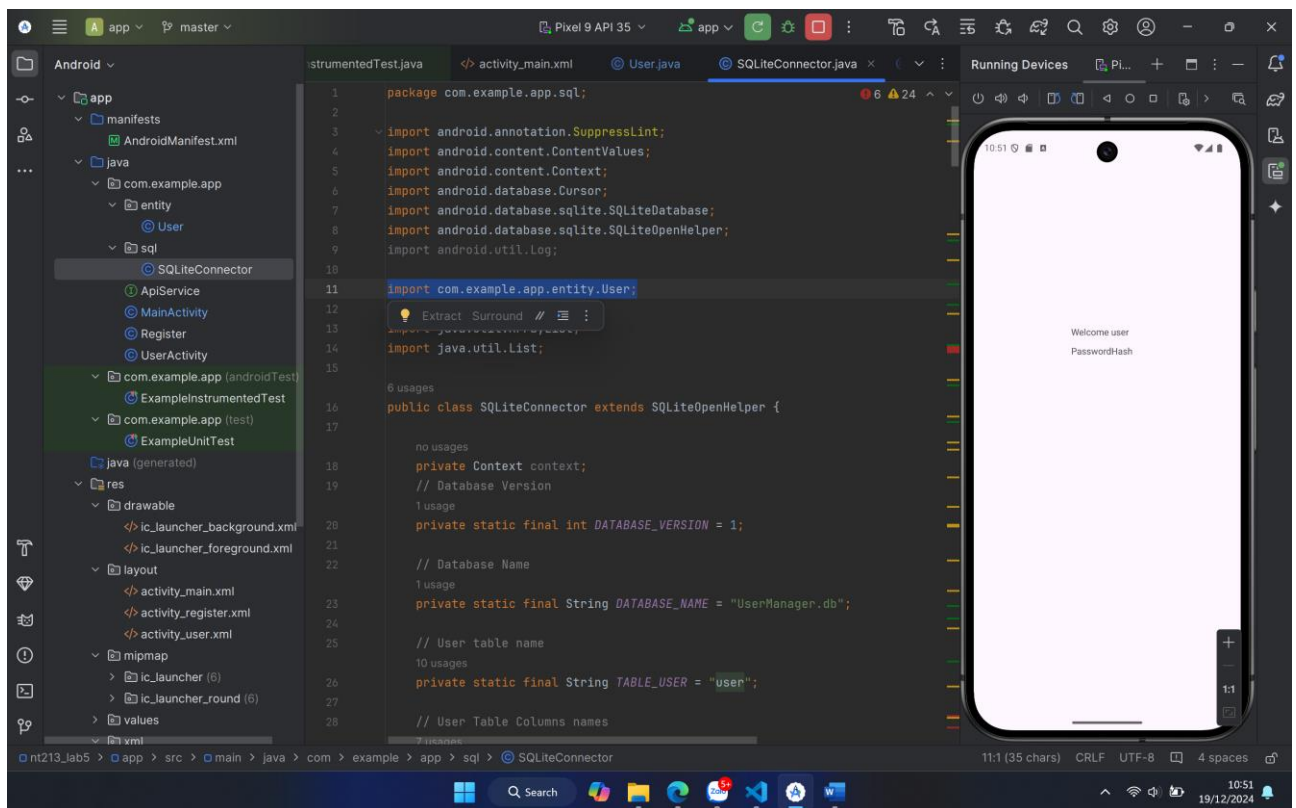
- Ta thực hiện sao chép file SQLiteConnector được cung cấp sẵn và dán tại mục `java/com.example.lab5_web`



- Định nghĩa class `User` gồm các trường `id`, `username`, `password` và `email` và các hàm `get` `set` tương ứng giá trị của các trường này.



- Import class User đến SQLiteConnector



- Coding cho chức năng đăng nhập đăng ký của ứng dụng:

- Chức năng đăng nhập:

```
bt_login.setOnClickListener(v -> {
    String email = et_email.getText().toString().trim();
    String password = et_password.getText().toString().trim();

    String hashPassword;
    try {
        MessageDigest messageDigest = MessageDigest.getInstance("SHA-256");
        byte[] bytes = messageDigest.digest(password.getBytes(StandardCharsets.UTF_8));
        hashPassword = ByteToHexConverter(bytes);
    } catch (NoSuchAlgorithmException e) {
        throw new RuntimeException(e);
    }

    if (!email.isEmpty() && Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
        if (!password.isEmpty()) {
            DBHelper dbHelper = new DBHelper(context: Login.this);
            boolean check = dbHelper.VerifyUser(email, hashPassword);
            if (check) {
                Intent intent = new Intent(context: Login.this, MainActivity.class);
                startActivity(intent);
            }
            else {
                Toast.makeText(context: Login.this, text: "Wrong email or password", Toast.LENGTH_SHORT).show();
            }
        }
    }
}
```

```
        else {
            Toast.makeText(context: Login.this, text: "Wrong email or password", Toast.LENGTH_SHORT).show();
        }
    }
    else {
        et_password.setError("Password can't be empty");
    }
}
else {
    et_email.setError("Your email is empty or have an incorrect format");
}
});
bt_signup.setOnClickListener(v -> {
    Intent intent = new Intent(context: Login.this, SignUp.class);
    startActivity(intent);
});
}
```

- Click vào button Login, nếu thông tin đầy đủ và tài khoản tồn tại, khớp với tài khoản ở database. Ta thực hiện truyền giá trị username vào Intent sau đó mở đến Activity sample_application cùng nội dung hello + username.
 - o Chức năng đăng ký:

```

void CreateUser() {
    UserModel userModel;
    String username = et_username.getText().toString().trim();
    String school = et_school.getText().toString();
    String class_room = et_class.getText().toString();
    String email = et_email.getText().toString().trim();
    String phone_number = et_phone_number.getText().toString().trim();
    String password = et_password.getText().toString().trim();
    String confirm_password = et_confirm_password.getText().toString().trim();

    if (username.isEmpty() || school.isEmpty() || class_room.isEmpty() || email.isEmpty() || password.isEmpty() || phone_number.isEmpty()) {
        Toast.makeText(context: SignUp.this, text: "Please fill all fields", Toast.LENGTH_SHORT).show();
    } else if (!Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
        et_email.setError("Incorrect email format");
    } else if (password.length() < 6) {
        et_password.setError("Password must be at least 6 character");
    } else if (!password.equals(confirm_password)) {
        et_confirm_password.setError("Password are not matching");
    } else {
        String hashPassword;
        try {
            MessageDigest messageDigest = MessageDigest.getInstance("SHA-256");
            byte[] bytes = messageDigest.digest(password.getBytes(StandardCharsets.UTF_8));
            hashPassword = ByteToHexConverter(bytes);
        } catch (NoSuchAlgorithmException e) {
            throw new RuntimeException(e);
        }
    }
}

```

```

DBHelper dbHelper = new DBHelper(context: SignUp.this);
userModel = new UserModel(id: 1, username, email, school, class_room, phone_number, hashPassword);
boolean success = dbHelper.AddUser(userModel);
Toast.makeText(context: SignUp.this, text: "Success=" + success, Toast.LENGTH_SHORT).show();
Intent intent = new Intent(packageContext: SignUp.this, Login.class);
startActivity(intent);
finish();
}
}

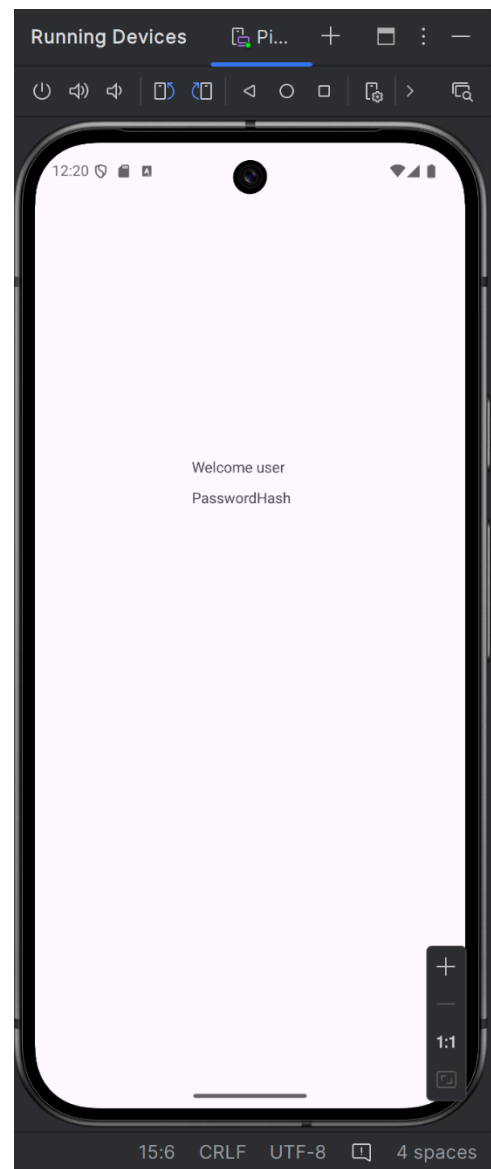
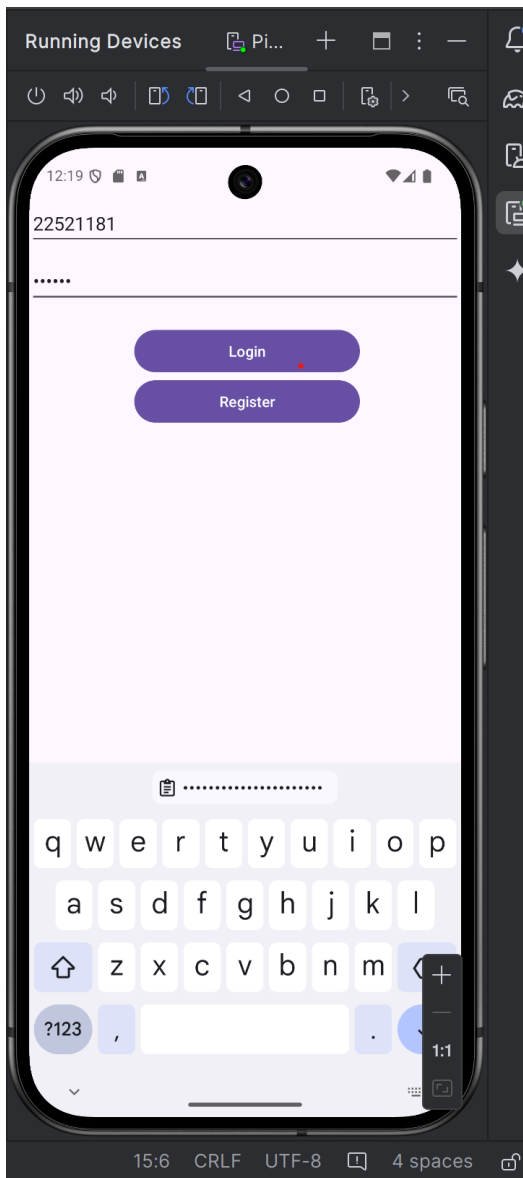
```

```

1 usage
String ByteToHexConverter(byte[] bytes) {
    StringBuilder sb = new StringBuilder();
    for (byte aByte : bytes) {
        sb.append(Integer.toString((aByte & 0xff) + 0x100, radix: 16).substring(beginIndex: 1));
    }
    return sb.toString();
}
}

```

- Khi click vào button Register, ứng dụng yêu cầu người dùng nhập đầy đủ thông tin cả các trường username, email, school, class_room, phone_number và password. Nếu có trường để trống thì hiện thông báo. Khi đã nhập đủ tiến hành sử dụng phương thức checkUser, nếu check không tồn tại username như đã nhập, thực hiện addUser vào DB và thông báo đăng ký thành công.
- Sau khi đăng nhập thành công sẽ hiển thị:



Yêu cầu 4 Điều chỉnh mã nguồn để password được lưu và kiểm tra dưới dạng mã hash thay vì plaintext.

Bài làm:

```
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.Objects;
```

- Import 2 thư viện trên để thực hiện hash mật khẩu.

```
String hashPassword;
try {
    MessageDigest messageDigest = MessageDigest.getInstance("SHA-256");
    byte[] bytes = messageDigest.digest(password.getBytes(StandardCharsets.UTF_8));
    hashPassword = ByteToHexConverter(bytes);
} catch (NoSuchAlgorithmException e) {
    throw new RuntimeException(e);
}

if (!email.isEmpty() && Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
    if (!password.isEmpty()) {
        DBHelper dbHelper = new DBHelper(context: Login.this);
        boolean check = dbHelper.VerifyUser(email, hashPassword);
        if (check) {
            Intent intent = new Intent(packageContext: Login.this, MainActivity.class);
            startActivity(intent);
        }
        else {
            Toast.makeText(context: Login.this, text: "Wrong email or password", Toast.LENGTH_SHORT)
        }
    }
    else {
        et_password.setError("Password can't be empty");
    }
}
else {
```

```
et_email.setError("Your email is empty or have an incorrect format");
}
```

- Ta tạo thêm phương thức hashPassword. Sử dụng thuật toán SHA 256.
- Ở chức năng đăng nhập, tiến hành phương thức check user với mật khẩu hash.

```
String hashPassword;
try {
    MessageDigest messageDigest = MessageDigest.getInstance("SHA-256");
    byte[] bytes = messageDigest.digest(password.getBytes(StandardCharsets.UTF_8));
    hashPassword = ByteToHexConverter(bytes);
} catch (NoSuchAlgorithmException e) {
    throw new RuntimeException(e);
}

DBHelper dbHelper = new DBHelper(context: SignUp.this);
userModel = new UserModel(id: 1, username, email, school, class_room, phone_number, hashPassword);
boolean success = dbHelper.AddUser(userModel);
Toast.makeText(context: SignUp.this, text: "Success=" + success, Toast.LENGTH_SHORT).show();
Intent intent = new Intent(packageContext: SignUp.this, Login.class);
startActivity(intent);
finish();
}
}

1 usage
String ByteToHexConverter(byte[] bytes) {
    StringBuilder sb = new StringBuilder();
    for (byte aByte : bytes) {
        sb.append(Integer.toString((aByte & 0xff) + 0x100, radix: 16).substring(beginIndex: 1));
    }
    return sb.toString();
}
```

- Ở chức năng Đăng ký, sau khi đăng ký thành công sẽ tiến hành Hash mật khẩu của user và lưu vào database.

Yêu cầu 5 Tạo một cơ sở dữ liệu tương tự bên ngoài thiết bị, viết mã nguồn thực hiện kết nối đến CSDL này để truy vấn thay vì sử dụng SQLite.

- Tạo một database trên mysql:

```
create database dducktai;
use dducktai;

CREATE TABLE test (
  fullname VARCHAR(255) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL
);
```

- Viết file Server.js cung cấp các API cơ bản để đăng ký và đăng nhập người dùng, đồng thời kết nối với cơ sở dữ liệu MySQL để lưu trữ và truy xuất thông tin người dùng:

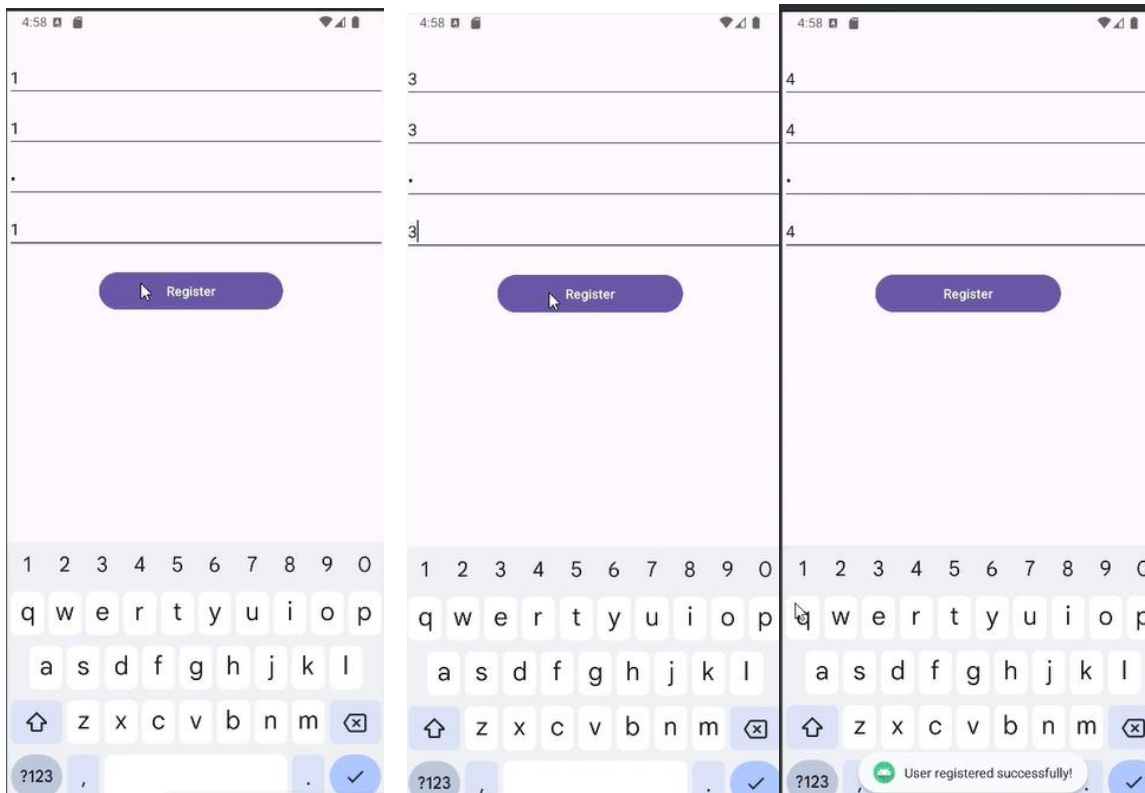
```
1 const express = require('express');
2 const bodyParser = require('body-parser');
3 const bcrypt = require('bcrypt');
4
5 const app = express();
6 const PORT = 8080;
7
8 // Middleware để parse JSON
9 app.use(bodyParser.json());
10
11 // Lưu trữ dữ liệu người dùng tạm thời
12 const users = [];
13
14 // API đăng ký người dùng
15 app.post('/api/v1/user/register', (req, res) => {
16   const { name, email, password } = req.body;
17
18   if (!name || !email || !password) {
19     return res.status(400).json({ message: 'All fields are required!' });
20   }
21
22   // Kiểm tra email đã tồn tại
23   const existingUser = users.find(user => user.email === email);
24   if (existingUser) {
25     return res.status(400).json({ message: 'Email is already registered!' });
26   }
27
28   // Thêm người dùng mới
29   const newUser = { name, email, password };
30   users.push(newUser);
31
32   // Hiển thị thông tin người dùng trong CMD
33   console.log('New user added:', newUser);
34
35   return res.status(201).json({ message: 'User registered successfully!', user: newUser });
36 });
37
38 // API kiểm tra (health check)
39 app.get('/', (req, res) => {
40   res.send('Server is running!');
41 });
42
43 // API kiểm tra (health check)
44 app.get('/', (req, res) => {
45   res.send('Server is running!');
46 });
47
48 // API đăng nhập người dùng
49 app.post('/api/v1/user/login', async (req, res) => {
50   const { name, password } = req.body;
51
52   const user = users.find(u => u.name === name);
53   if (user && await bcrypt.compare(password, user.password)) {
54     res.status(200).json({ name: user.name, email: user.email });
55   } else {
56     res.status(401).json({ message: 'Invalid username or password' });
57   }
58 });
59
60 // Bắt đầu server
61 app.listen(PORT, () => {
62   console.log('Server is running on http://localhost:${PORT}');
63 });
```

- Run file để cho phép kết nối:

```
C:\Windows\System32\cmd.exe - node server.js
Microsoft Windows [Version 10.0.26120.1252]
(c) Microsoft Corporation. All rights reserved.

C:\Code\Bao mat web va ung dung - NT213.P11.ANTT\Lab\W5\final\backend>node server.js
Server is running on http://localhost:8080
```

- Mở build app và đăng kí tài khoản bất kì:



- Kết quả:

```
C:\Windows\System32\cmd.exe - node server.js
Microsoft Windows [Version 10.0.26120.1252]
(c) Microsoft Corporation. All rights reserved.

C:\Code\Bao mat web va ung dung - NT213.P11.ANTT\Lab\W5\final\backend>node server.js
Server is running on http://localhost:8080
New user added: {
  name: '1',
  email: '1',
  password: '$2a$10$3mL1pHx1v25Wjb2NhI130u0C1Ka88opcZME/KPwY5p0ucG1aDZfr6'
}
New user added: {
  name: '3',
  email: '3',
  password: '$2a$10$hUFCVrQRKp8wxU2IfrahMeMjzh1Pvb0nq8iQdtyVqfNrzi7A6VPMa'
}
New user added: {
  name: '4',
  email: '4',
  password: '$2a$10$TnG2X2vNwzW5KeChcFfm80FqJhAE/4vhtxhRGKDikrmVdnqXoY6e'
}
```

Yêu cầu 6 Với ứng dụng đã xây dựng, tìm hiểu và sử dụng công cụ ProGuard để tối ưu hóa mã nguồn. Trình bày khác biệt trước và sau khi sử dụng?

1. ProGuard là gì?

Proguard một java tool miễn phí trong Android, giúp chúng ta thực hiện các thao tác sau:

- Thu nhỏ (Giảm thiểu) code: Loại bỏ mã không sử dụng trong dự án.
- Obfuscate code: Đổi tên tên của lớp, trường, ...
- Tối ưu hóa code: Thực hiện những việc như nội tuyến các hàm. Nói tóm lại, Proguard tạo ra tác động sau đây cho dự án:
- Làm giảm kích thước của ứng dụng.
- Loại bỏ các lớp và phương thức không sử dụng góp phần vào phương thức đếm giới hạn 64k method của ứng dụng Android.
- Giúp cho ứng dụng khó bị dịch ngược hơn bằng cách làm xáo trộn mã.

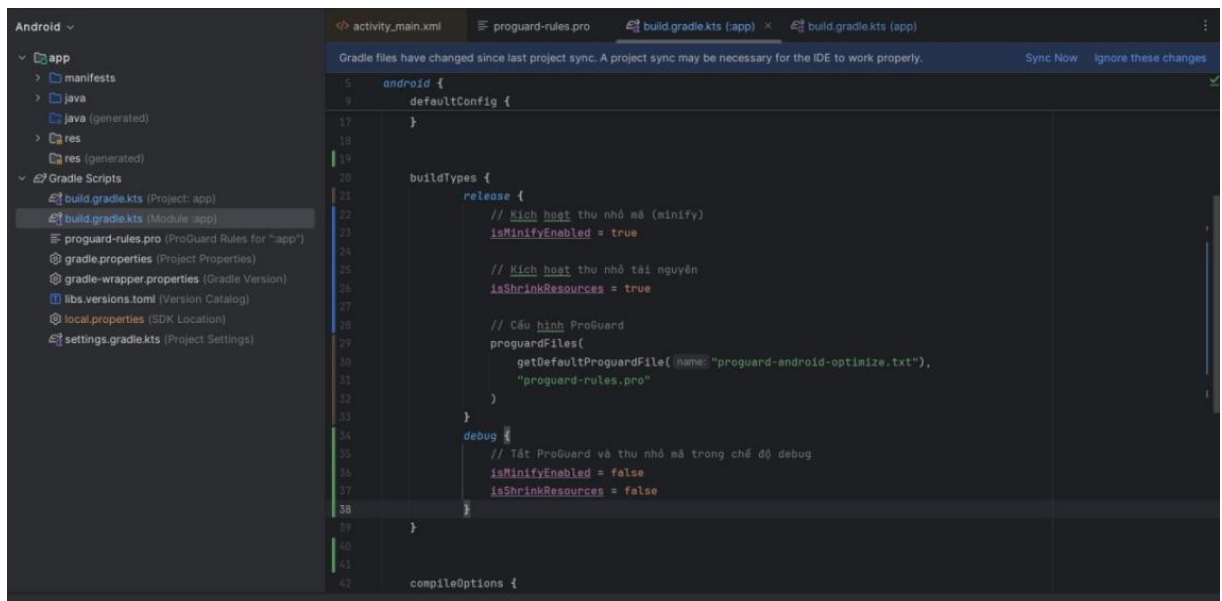
Trong Android, proguard rất hữu ích để tạo ra một ứng dụng production-ready. Nó giúp chúng ta giảm mã và làm cho ứng dụng chạy nhanh hơn. Proguard được khởi tạo mặc định trong Android Studio và nó giúp chúng ta theo nhiều cách, một vài trong số chúng được đề cập dưới đây:

Nó làm xáo trộn mã, có nghĩa là nó thay đổi tên các class thành một cái tên mới có số chữ cái ít hơn. Ví dụ như MainViewModel nó có thể đổi tên thành A. Kỹ thuật đảo ngược ứng dụng của bạn sẽ trở thành một nhiệm vụ khó khăn sau khi proguard làm xáo trộn ứng dụng.

Nó thu nhỏ tài nguyên, tức là bỏ qua các tài nguyên không được gọi bởi các tệp class của chúng ta, không được sử dụng trong ứng dụng Android của chúng ta như hình ảnh từ drawables, ... Điều này sẽ làm giảm kích thước ứng dụng rất nhiều. Bạn luôn luôn cần chú ý tối ưu hóa ứng dụng của mình để giữ cho nó nhẹ và nhanh.

2. Cấu hình

- Tại build-gradle.kts sửa lại buildTypes:



- Cấu hình cho file proguard-rules.pro

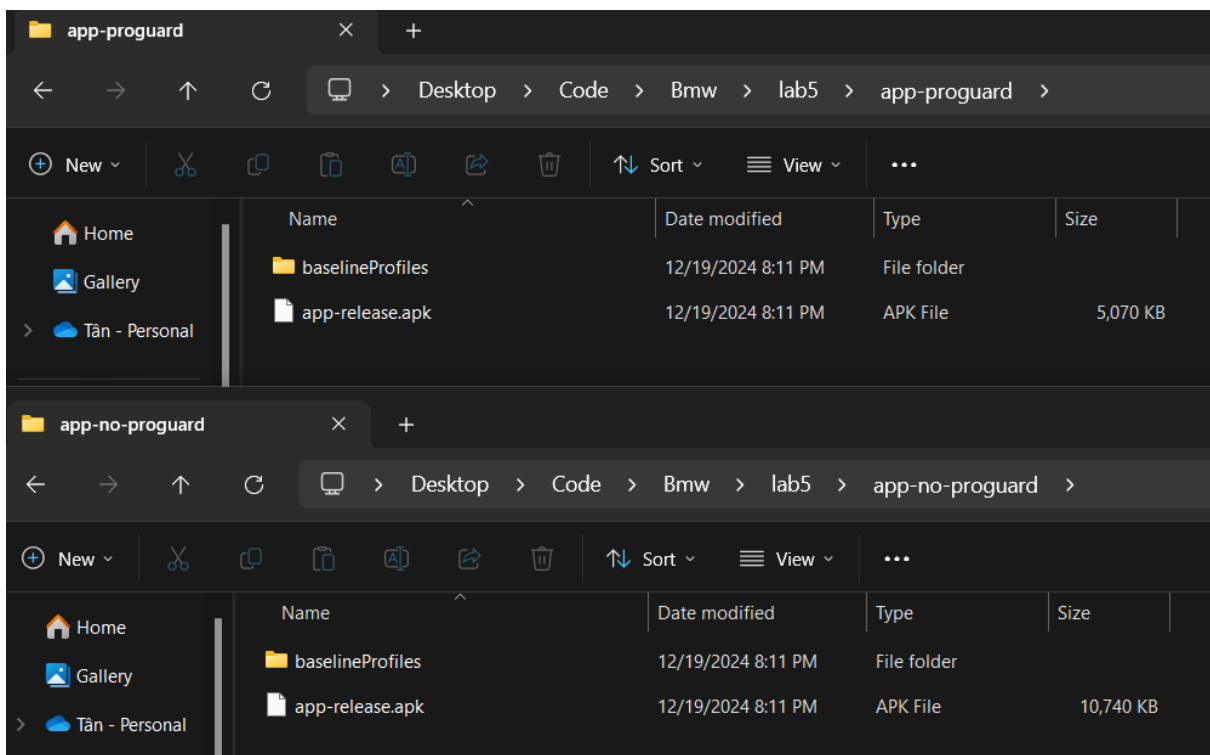


– Khi Sử Dụng ProGuard

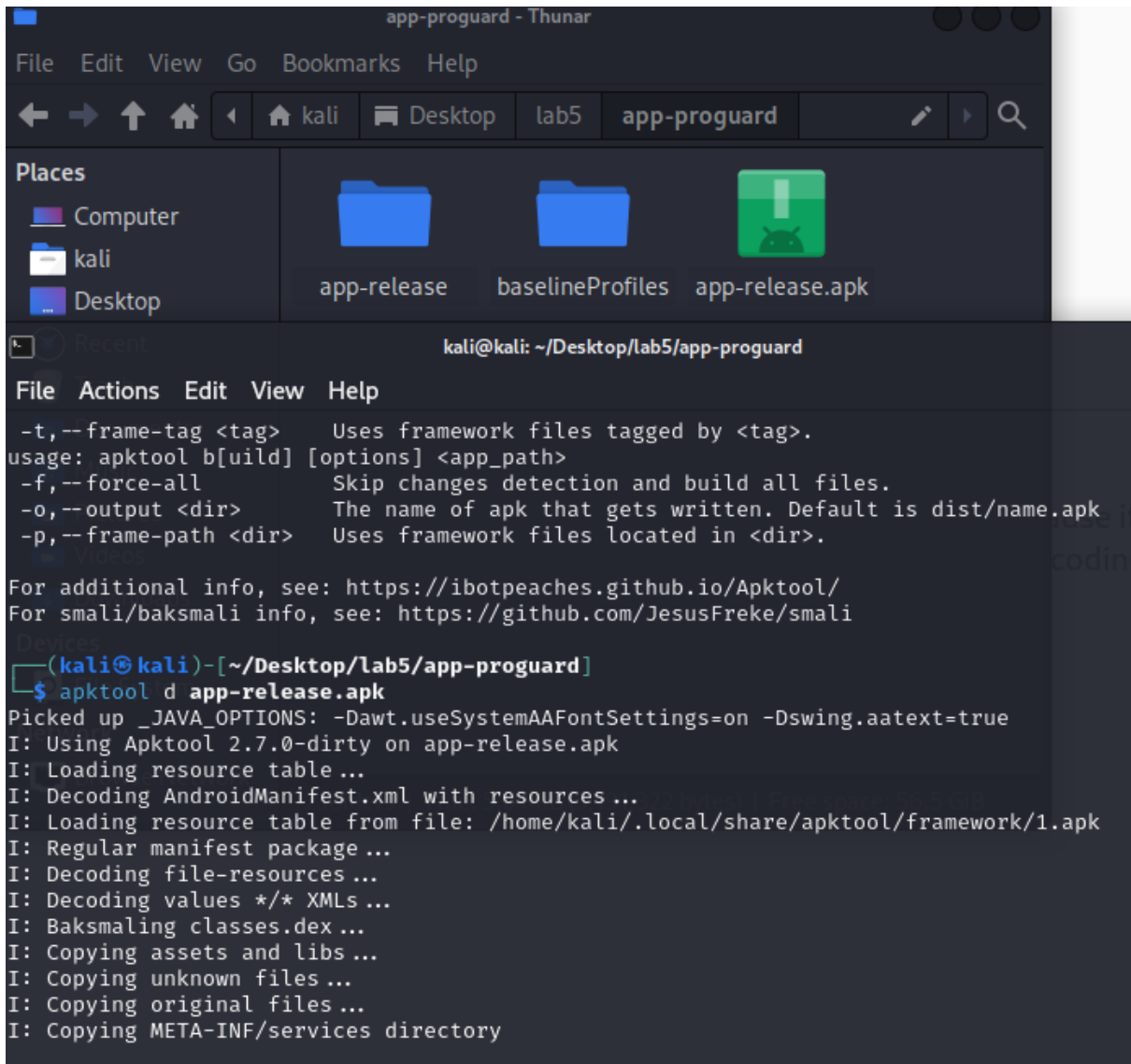
1. **Giảm Kích Thước Ứng Dụng:** ProGuard loại bỏ mã không sử dụng và tối ưu hóa các lớp và phương thức để giảm kích thước ứng dụng.
2. **Bảo Vệ Mã Nguồn:** ProGuard làm khó khăn việc phân tích ngược bằng cách rút gọn tên lớp, phương thức và biến. Điều này làm cho mã nguồn khó hiểu hơn nếu bị dịch ngược.
3. **Tối Ưu Hiệu Suất:** Một số tối ưu hóa có thể làm cho ứng dụng chạy hiệu quả hơn, mặc dù điều này không luôn luôn đáng kể.
4. **Kết Hợp Mã:** Nó có thể kết hợp các lớp và phương thức để tạo ra mã ngắn gọn và hiệu quả hơn.

– Khi Không Sử Dụng ProGuard

1. **Dễ Dàng Bị Phân Tích Ngược:** Mã nguồn sẽ dễ dàng hơn để phân tích và hiểu bởi những người không có quyền truy cập hợp pháp.
2. **Kích Thước Ứng Dụng Lớn Hơn:** Mã không sử dụng sẽ không được loại bỏ, dẫn đến kích thước ứng dụng lớn hơn.
3. **Hiệu Suất Không Tối Ưu:** Ứng dụng có thể chứa các mã thừa không cần thiết, dẫn đến hiệu suất kém hơn so với khi sử dụng ProGuard.
4. **Dễ Dàng Debug:** Việc không sử dụng ProGuard làm cho việc gỡ lỗi dễ dàng hơn vì không có sự rút gọn tên và tối ưu hóa mã.

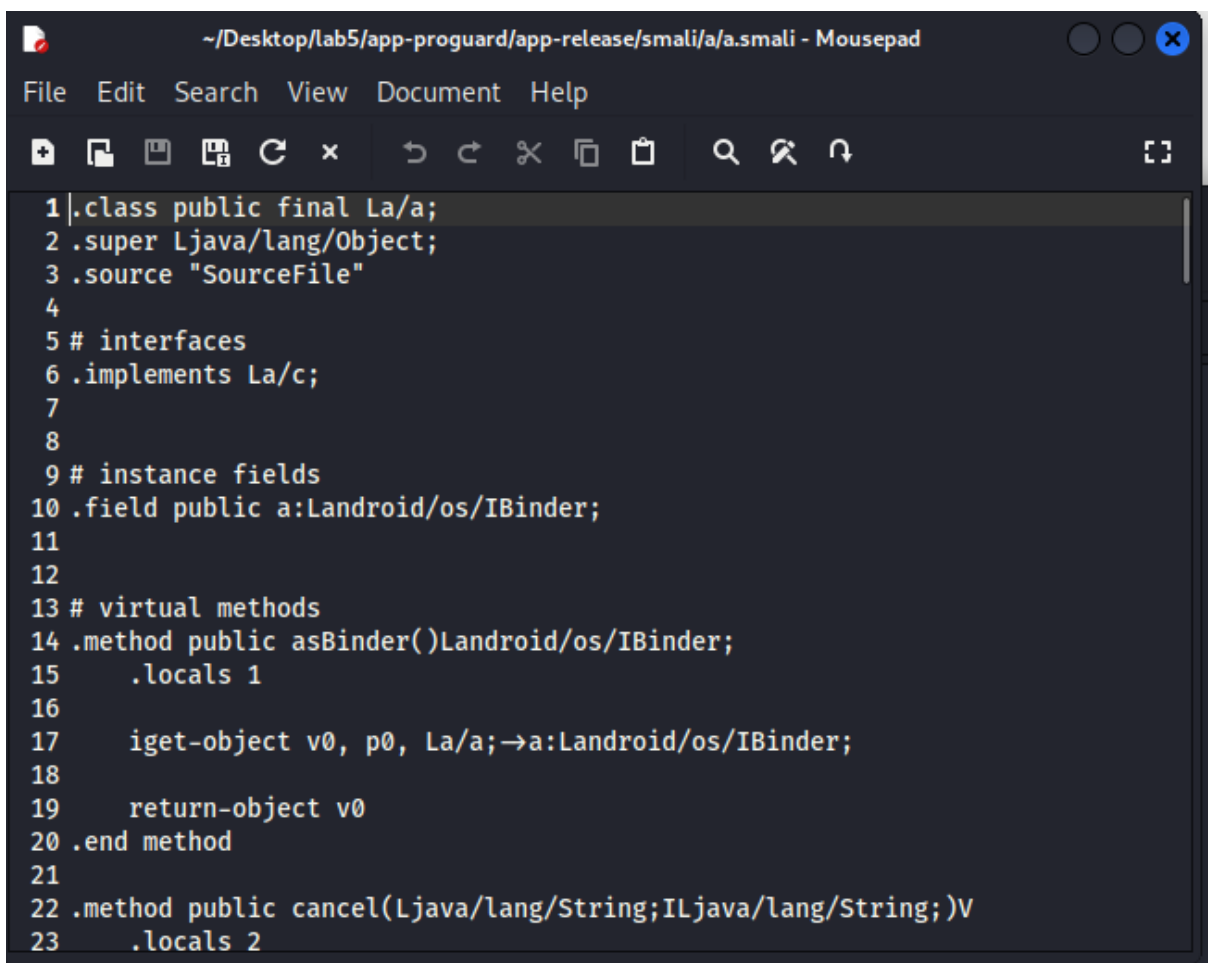
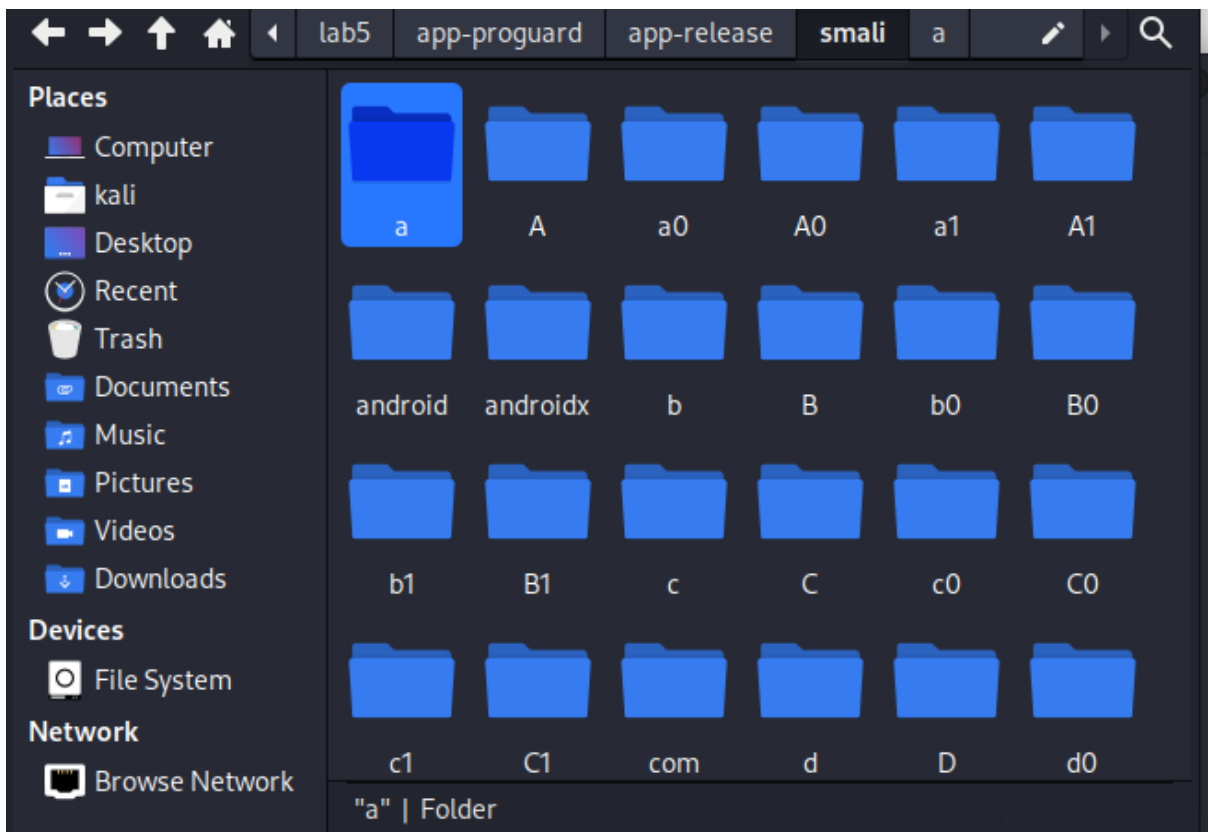


- Dùng apktool để dịch ngược app có dùng proguard:



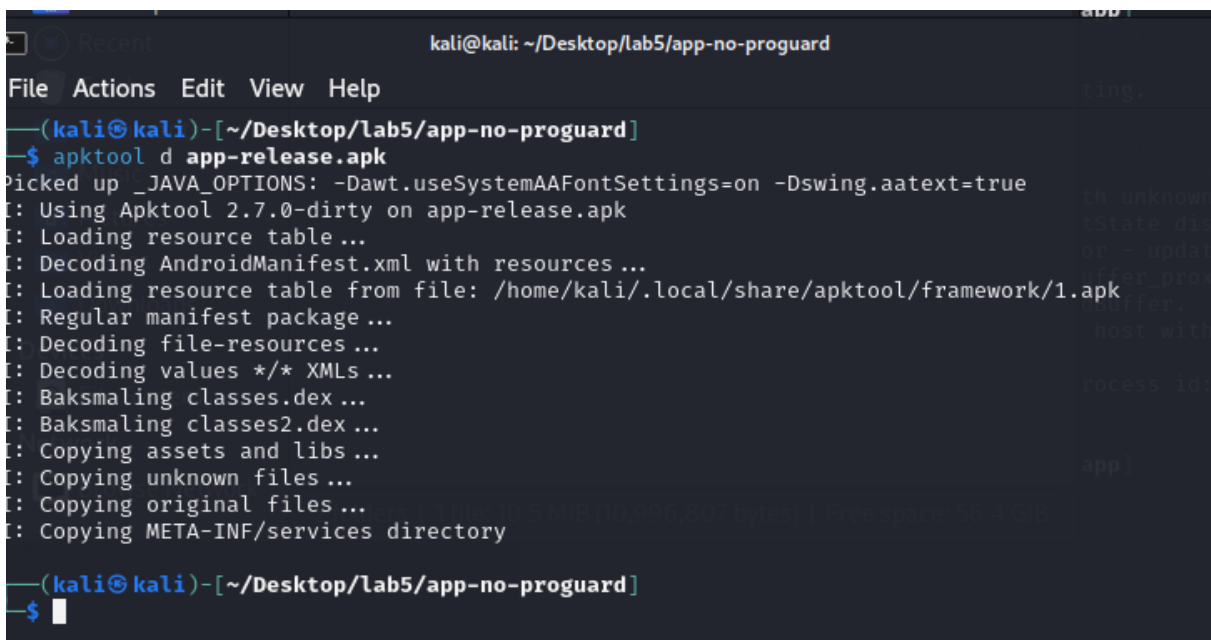
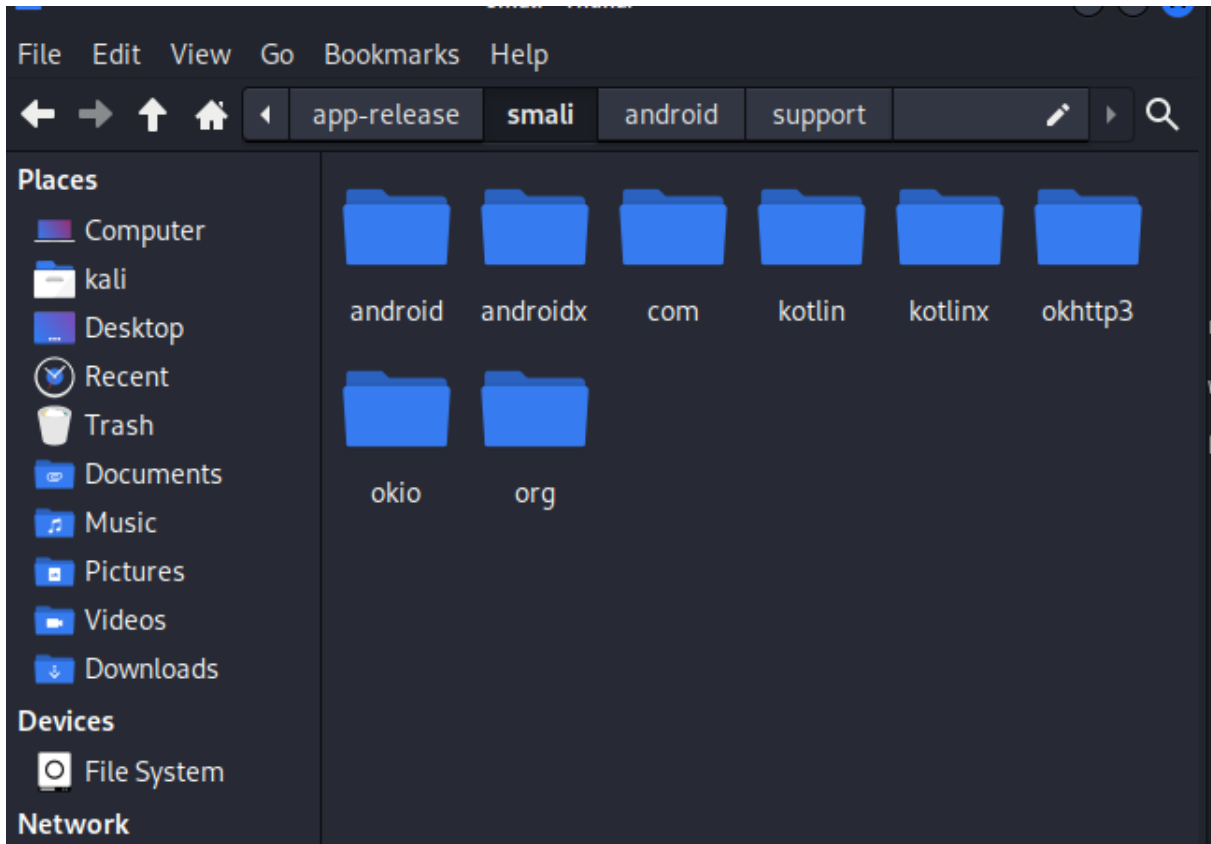
The screenshot shows a Kali Linux terminal window titled "app-proguard - Thunar". The terminal output displays the usage of the apktool command and the successful decompilation of the app-release.apk file. The terminal text is as follows:

```
File Edit View Go Bookmarks Help
kali Desktop lab5 app-proguard
Places
Computer
kali
Desktop
app-release baselineProfiles app-release.apk
kali@kali: ~/Desktop/lab5/app-proguard
File Actions Edit View Help
-t,--frame-tag <tag> Uses framework files tagged by <tag>.
usage: apktool b[uild] [options] <app_path>
-f,--force-all Skip changes detection and build all files.
-o,--output <dir> The name of apk that gets written. Default is dist/name.apk
-p,--frame-path <dir> Uses framework files located in <dir>.
For additional info, see: https://ibotpeaches.github.io/Apktool/
For smali/baksmali info, see: https://github.com/JesusFreke/smali
(kali@kali)-[~/Desktop/lab5/app-proguard]
$ apktool d app-release.apk
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on -Dswing.aatext=true
I: Using Apktool 2.7.0-dirty on app-release.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources...
I: Loading resource table from file: /home/kali/.local/share/apktool/framework/1.apk
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmaling classes.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...
I: Copying META-INF/services directory
```



Đọc code khó hiểu và nhiều file phức tạp

App không dùng proguard



App dễ đọc dẫn đến nguy cơ dễ bị dịch ngược

--HẾT--